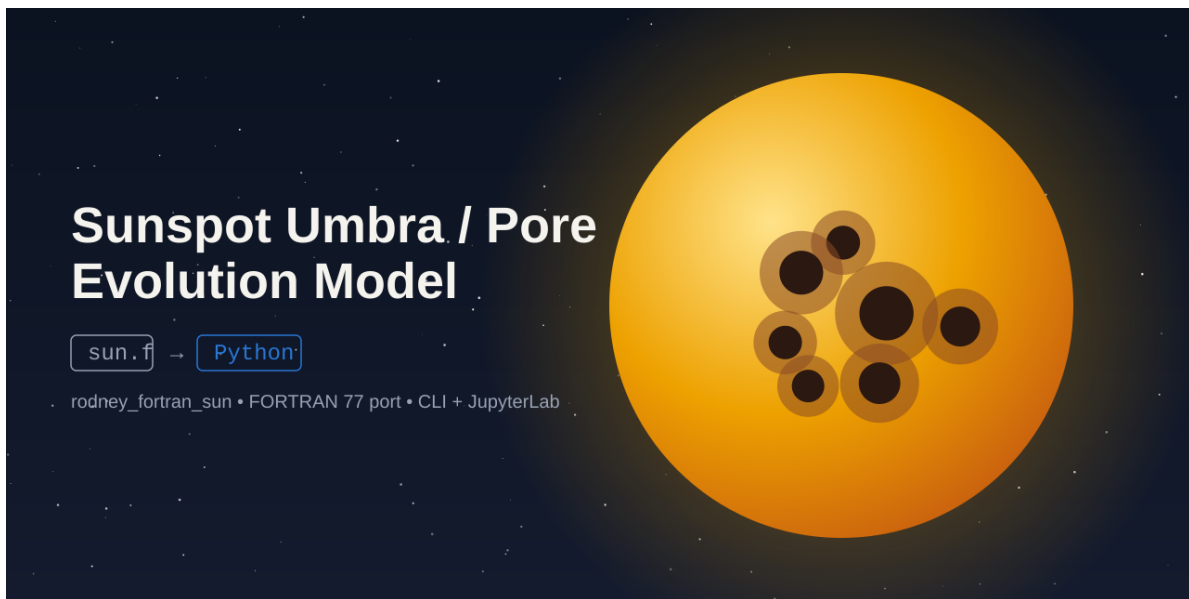


Sunspot Umbra / Pore Evolution Model

Analysis report — the original FORTRAN model and its validated Python port

David J Jackson

2026-08-27



Executive summary

`sun.f` is a FORTRAN 77 program Rodney wrote to model how sunspot **umbra** (dark core) and **penumbra** (surrounding lighter region) evolve day to day, driven by real observations from the Royal Greenwich Observatory. It has been ported to a Python package, `sunmodel`, with a command-line interface and a JupyterLab notebook wrapper.

This report:

1. summarizes what the original model computes and how,
2. runs the Python port end-to-end against the real input data, and

3. **verifies live, in this document**, that the port reproduces the original gfortran output exactly, row for row.

See `SUN_F_ANALYSIS.pdf` in this repository for an exhaustive, line-by-line walkthrough of the original FORTRAN source.

Background: what the model computes

The program combines two loosely-coupled sub-models:

- **An empirical, saturating growth curve** (`EXPO/ENERGY/MASS/JTEMP/E2` in the original; `sunmodel.empirical` in the port) — a hand-fit function of elapsed time with no documented physical derivation, intended to estimate a likelihood of pore evolution.
- **A small SEIR (Susceptible/Exposed/Infected/Removed) submodel** over 8 solar-disk “polygon” regions (`SPUR/ZCELLS/MOVES` in the original; `sunmodel.seir` in the port) — borrowed wholesale from an epidemiological model and repurposed to represent how “infection” (spot state) might spread between regions.

Both feed a per-day estimate of umbra (`SUSQI`) and sunspot/pore count (`SUSQS`), computed record-by-record from daily observations and written to a CSV report. The user supplies five calibration values up front: the number of umbra seen, total sunspots seen, and three percentages (umbra-with-penumbra, sunspots-in-umbra, umbra-area-to-group-area).

i An important caveat, found while porting

Tracing the original control flow shows the SEIR/`EXPO` branch only ever executes **once**, on the very first data record of the entire run (`K=1` is hardcoded, and the loop counter it gates never resets across batches). Worse, the line that would apply `EXPO`’s result to the output (`POSQ=F*POSQ`) is commented out in `sun.f` — so that call’s result is computed but never used. In practice, the numbers in `SUN.CSV` are driven almost entirely by the simpler `1/IREC` decay term and the raw per-record `ICUA/ICWSA` values, not by the more elaborate curve-fit or SEIR machinery. The Python port reproduces this behavior exactly (including calling `expo()` and discarding its result) for numerical fidelity, but it’s worth knowing when interpreting the results below.

Data

File	Role
<code>rgo_data.prn</code>	Daily sunspot group observations (group id, day, whole-spot area, umbra area, ...)

File	Role
polypara.dat	Initial S/E/I/R state and rate constants for the 8 SEIR “polygon” regions
SUN.CSV	Reference output from a real <code>gfortran</code> compile-and-run of <code>sun.f</code> , checked into this repo

The Python port

```
import pandas as pd
import matplotlib.pyplot as plt

from sunmodel import RunConfig, SunReport, SunSimulation, read_report_rows
```

`sunmodel` mirrors the original’s structure module-for-module:

Module	Ports
<code>empirical.py</code>	EXPO, ENERGY, MASS, JTEMP, E2
<code>seir.py</code>	SPUR, ZCELLS, MOVES, <code>ran2</code> (replaced with <code>random.Random</code>)
<code>rgo_data.py</code>	the <code>rgo_data.prn</code> reader
<code>report.py</code>	the <code>SUN.CSV</code> writer
<code>simulation.py</code>	the main batch-driven loop
<code>cli.py</code> / <code>main.py</code>	the interactive prompts

The unused polynomial-fit arrays (`Y`, `T`, `ETA`, ...) from the original’s dead code path were dropped; everything else was ported with intentional fidelity, including the operator-precedence quirk and dead `EXPO` call noted above.

Live demonstration

Running the port against the real data files, with the same five parameters recorded in the checked-in `SUN.CSV` (4 umbra seen, 27 total sunspots, `delta`=0.40, `gamma`=0.60, `P`=0.30):

```
import tempfile
from pathlib import Path
```

```

config = RunConfig(
    n_umbra_seen=4, n_sunspots_seen=27,
    penumbra_pct=0.40, umbra_sunspot_pct=0.60, umbra_group_area_pct=0.30,
)

scratch_csv = Path(tempfile.mkstemp(suffix=".csv")[1])
with SunReport(scratch_csv) as report:
    sim = SunSimulation(config, "rgo_data.prn", "polypara.dat", report, seed=0)
    while sim.run_batch():
        pass

npores, nspots = sim.pore_summary()
print(f"Sunspots = {sim.total_sunspots}    Pores = {npores}    Actual sunspots = {nspots}")

```

Sunspots = 27 Pores = 11 Actual sunspots = 16

```

results = pd.DataFrame(read_report_rows(scratch_csv))
pd.concat([results.head(5), results.tail(5)])

```

Table 1: Per-day model output (first and last 5 rows)

	day	umbra	pores	cwsa	cua
0	1	0.6731	8.1978	10	50
1	1	15.3357	2.0628	2	10
2	1	10.9541	10.3955	14	70
3	1	8.5198	10.3970	14	70
4	1	6.9708	24.1886	93	550
77	30	0.4884	11.3507	16	80
78	30	0.4823	12.2212	18	90
79	30	0.4763	6.9168	8	40
80	30	0.4704	20.4072	52	310
81	30	0.4647	5.4877	6	30

```

# Colors are slots 1 (blue) and 2 (orange) of the project's validated
# categorical palette -- fixed hue order, not cycled, single shared axis
# since both series are the same kind of derived quantity.
UMBRA_COLOR = "#2a78d6"
PORES_COLOR = "#eb6834"

fig, ax = plt.subplots()

```

```

x = range(1, len(results) + 1)
ax.plot(x, results["umbra"], color=UMBRA_COLOR, linewidth=2, label="Umbra")
ax.plot(x, results["pores"], color=PORES_COLOR, linewidth=2, label="Pores")
ax.set_xlabel("Record #")
ax.set_ylabel("Estimated value")
ax.set_title("Umbra vs. Pores evolution")
ax.grid(True, color="#dddddd", linewidth=0.7)
for spine in ("top", "right"):
    ax.spines[spine].set_visible(False)
ax.legend(frameon=False)
fig.tight_layout()
plt.show()

```

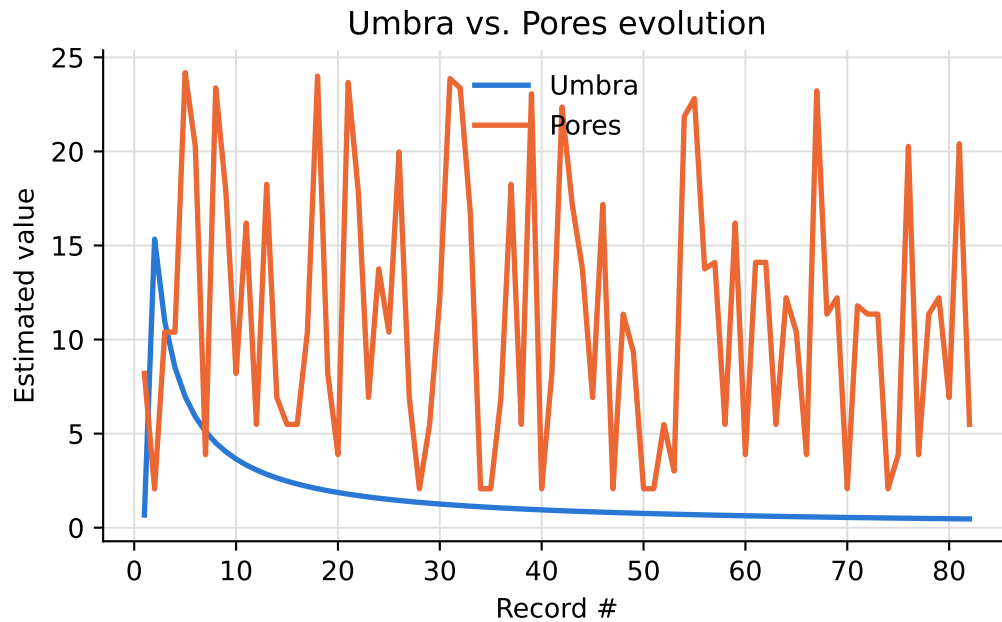


Figure 1: Umbra vs. Pores evolution across all 82 observed records

Umbra (blue) rises briefly then decays smoothly — the expected shape of the $1/\text{IREC}$ term that dominates after the first record. Pores (orange) is noisy record-to-record because SUSQS scales almost linearly with each record's raw observed umbra area, which itself varies a great deal group to group.

Validation

The claim that this port is faithful isn't just asserted — it's checked against the real `gfortran`-produced `SUN.CSV` right here, live:

```
reference_rows = read_report_rows("SUN.CSV")
ported_rows = read_report_rows(scratch_csv)

matches = reference_rows == ported_rows
print(f"Reference rows: {len(reference_rows)}")
print(f"Ported rows:    {len(ported_rows)}")
print(f"Byte-for-byte match: {matches}")

assert matches, "Ported output diverged from the reference FORTRAN output"
```

```
Reference rows: 82
Ported rows:    82
Byte-for-byte match: True
```

This same comparison is also encoded as a permanent regression test in `tests/test_simulation.py`, run on every change via `pytest`.

Discussion

- The model's headline behavior — a decaying “Umbra” estimate and a noisy “Pores” estimate that tracks raw observed spot area — is driven almost entirely by simple arithmetic on the input data (1/IREC decay, direct scaling by ICUA/ICWSA), **not** by the more sophisticated empirical curve-fit or SEIR submodels those variable names suggest. See the callout in [Background] above.
- The SEIR submodel (ZCELLS/SPUR) is fully ported and unit-tested, and produces sensible, bounded, finite output — but it currently only contributes to the very first record of a run, and even that contribution is a smaller POSQ term rather than a persistent evolving state (since ZCELLS is always called with `timestep=0`, it re-reads `polypara.dat` fresh each time rather than carrying state forward).

Update: the two behavioral recommendations below (SEIR/EXPO actually driving output, and a decay that doesn't depend on raw record count) are now implemented as an opt-in **robust** dynamics mode, alongside the original **legacy** mode described above. Nothing above this note changes for **legacy** mode — it remains byte-for-byte identical to `sun.f`'s output (see [Validation](#)) and stays the default in `sunmodel.cli` and `SunSimulation`, so existing workflows and the `SUN.CSV` comparison are unaffected unless `--dynamics robust` (or `dynamics="robust"`) is requested explicitly.

Recommendations / future work

- ~~Decide whether the SEIR/EXP0 machinery should actually drive output.~~ **Done, opt-in.** robust mode carries ZCELLS' polygon state forward across every record instead of reinitializing it, and applies EXP0's result to POSQ (the original's commented-out `POSQ=F*POSQ`) on every record rather than discarding it. See `SunSimulation._robust_posq` in `sunmodel/simulation.py` and `tests/test_simulation_robust.py`.
- ~~Consider replacing 1/IREC decay~~ **Done, opt-in.** robust mode's `remq` term now decays with elapsed *calendar* days since the run's first record (`SunSimulation._robust_remq`), rather than with how many records the process happened to have handled. This matters in practice: `rgo_data.prn`'s own `Day` column jumps from day 2 to day 26 partway through the bundled sample, a 24-day gap the old 1/IREC count couldn't see. A run started mid-dataset (or resumed) now reproduces the trajectory a continuous run would have from that point on. The half-life of that decay (default 14 days) has no physical derivation — same caveat as EXP0's curve — and is a constructor parameter (`remq_half_life_days`) for anyone who wants to tune it.
- **Run against the full historical RGO dataset**, not just the 82-record sample bundled here, to see the model's longer-run behavior. (*Still open — needs a larger RGO extract than what's checked into this repo.*)
- **Add a proper scientific validation** — compare model output against published sunspot umbra/penumbra ratios from the Bogdan et al. (2004) paper the original cites, beyond just confirming the port matches the original code's arithmetic. (*Still open — needs that paper's reference figures.*)

Appendix

- Source: [sun.f](#) · line-by-line walkthrough: [SUN_F_ANALYSIS.pdf](#)
- Python port: [sunmodel/](#) · tests: [tests/](#)
- Interactive notebook: [Sun_Model.ipynb](#)
- Repository: https://github.com/davidjayjackson/rodney_fortran_sun